

The Complete Maze-Generator Program

Let's begin by taking a look at the complete Python and JavaScript source code for the program, which uses the recursive backtracking algorithm for maze generation. The rest of this chapter explains each section of code individually.

Copy this Python code to a file named *mazeGenerator.py*:

Python

```
import random

WIDTH = 39 # Width of the maze (must be odd).
HEIGHT = 19 # Height of the maze (must be odd).
assert WIDTH % 2 == 1 and WIDTH >= 3
assert HEIGHT % 2 == 1 and HEIGHT >= 3
SEED = 1
random.seed(SEED)

# Use these characters for displaying the maze:
EMPTY = ' '
MARK = '@'
WALL = chr(9608) # Character 9608 is '■'
NORTH, SOUTH, EAST, WEST = 'n', 's', 'e', 'w'

# Create the filled-in maze data structure to start:
maze = {}
for x in range(WIDTH):
    for y in range(HEIGHT):
        maze[(x, y)] = WALL # Every space is a wall at first.

def printMaze(maze, markX=None, markY=None):
    """Displays the maze data structure in the maze argument. The
    markX and markY arguments are coordinates of the current
    '@' location of the algorithm as it generates the maze."""

    for y in range(HEIGHT):
        for x in range(WIDTH):
            if markX == x and markY == y:
                # Display the '@' mark here:
                print(MARK, end='')
            else:
                print(maze[(x, y)], end='')
            if x < WIDTH - 1:
                print(' ', end='')
        print()
```